

Algorithms and Data Structures I.

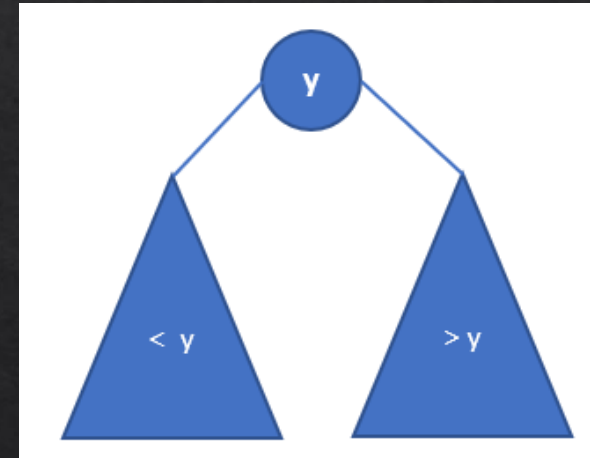
Practice 7.

Binary search tree

A **binary search tree (BST)** is a binary tree whose nodes each store a key, and the tree satisfies the **binary search tree property**, which states that:

- the key in each node must be greater than any key stored in its left subtree and
- less than any key stored in its right subtree.

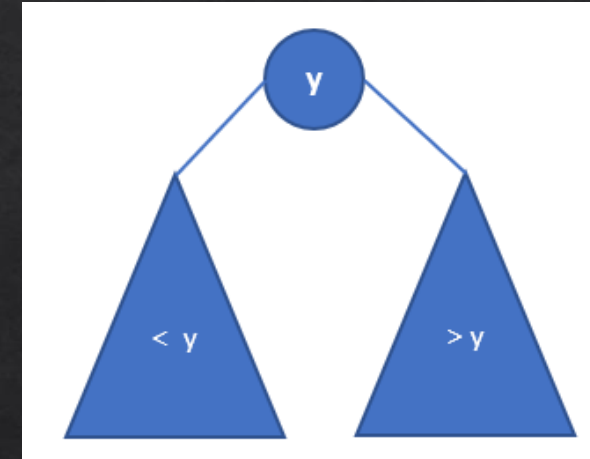
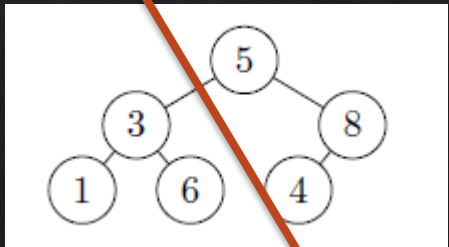
A binary sort tree is similar to a BST but may have equal keys. In a BST all keys must be different!



Binary search tree

The following definition is **not correct**: for every node the key of its left child is smaller, and the key of its right child is larger.

Counterexample: → this is not a BST!!!



Building a BST

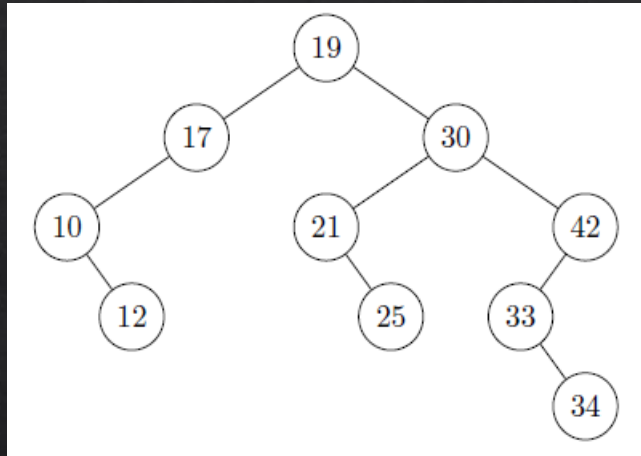
◆ Build a binary tree with the insertion of the following elements:

19, 17, 30, 10, 21, 12, 42, 25, 33, 34

Building a BST

◆ Build a binary tree with the insertion of the following elements:

19, 17, 30, 10, 21, 12, 42, 25, 33, 34



Reconstructing a BST from a traversal

- ◆ A Binary Search Tree can be reconstructed by its preorder or postorder traversal! Why?

Examples:

➤ From preorder:
2, 1, 6, 3, 5, 8, 7, 9

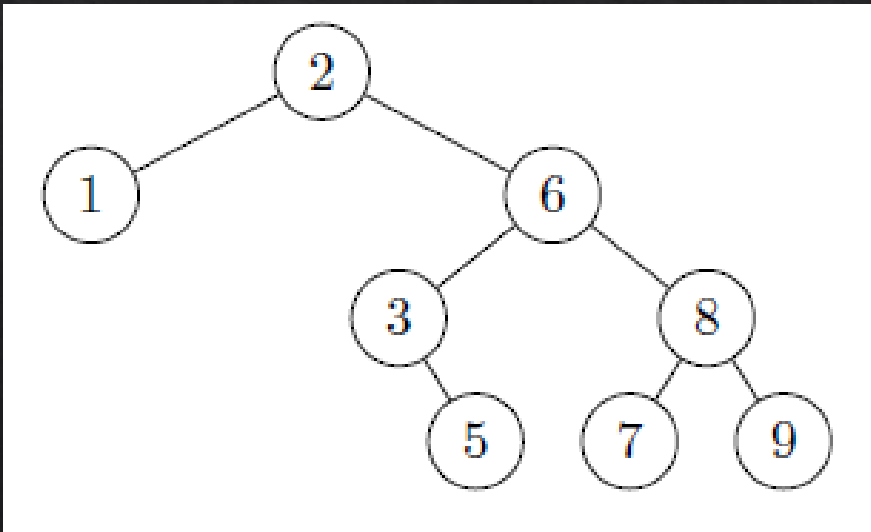
➤ From postorder:
2, 4, 3, 9, 8, 10, 7, 5

Reconstructing a BST from a traversal

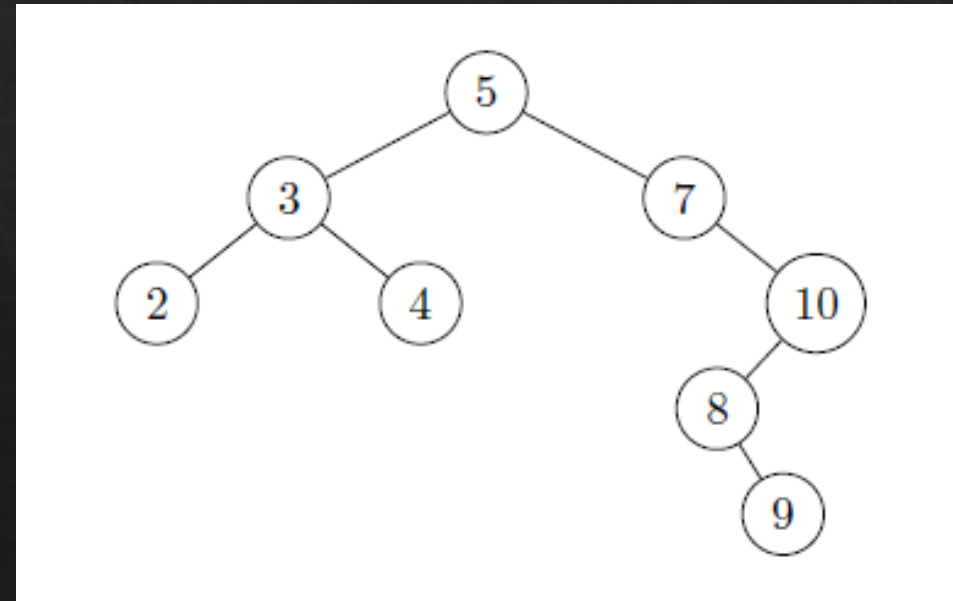
- ◆ A Binary Search Tree can be reconstructed by its preorder or postorder traversal! Why?

Examples:

- From preorder:
2, 1, 6, 3, 5, 8, 7, 9



- From postorder:
2, 4, 3, 9, 8, 10, 7, 5

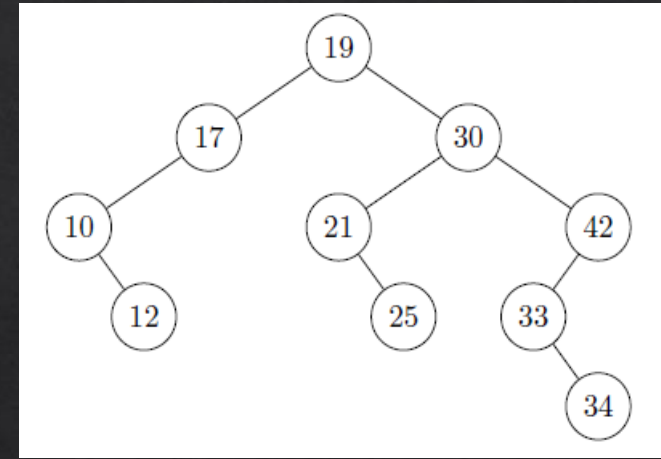


Basic operations of BSTs

- looking for a node with key value $k \rightarrow \text{search}(t, k)$
 - if there is no such node then it should return NIL
- inserting a node with key value $k \rightarrow \text{insert}(t, k)$
 - if there is a node with key value k then do not insert
- deleting a node with key value $k \rightarrow \text{del}(t, k)$
 - if there is no node with key value k then do nothing

Basic operations in BSTs

- ◊ Search for the keys 25, and 23!
- ◊ What keys can go in the left subtree of 21?
- ◊ What keys can go in the right subtree of 12?
- ◊ How can we delete 12? And 17? 42? 30?
- ◊ What happens if we traverse the tree in an inorder manner?



Basic operations in BSTs

◆ Search for the keys 25, and 23!

◆ What keys can go in the left subtree of 21?

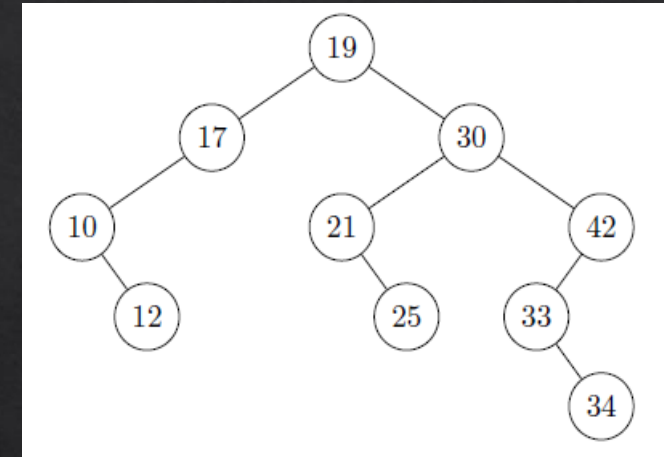
20

◆ What keys can go in the right subtree of 12?

13..16

◆ How can we delete 12? And 17? 42? 30?
we will see soon

◆ What happens if we traverse the tree in an inorder manner?



We see the keys in
ascending order

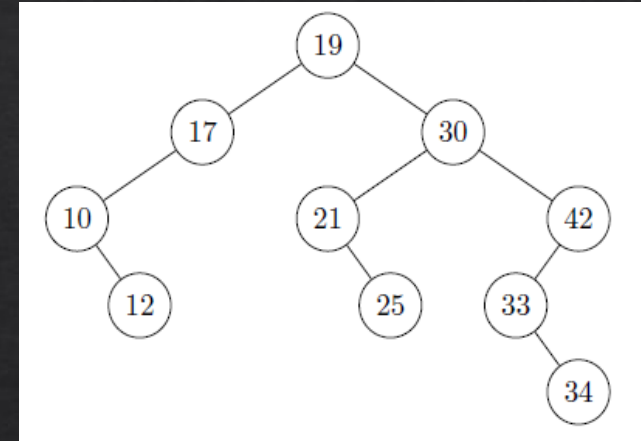
Search

searching for a node with key value k

◇ Recursive algorithm:

$\text{search}(t : \text{Node}^* ; k : \mathcal{T}) : \text{Node}^*$

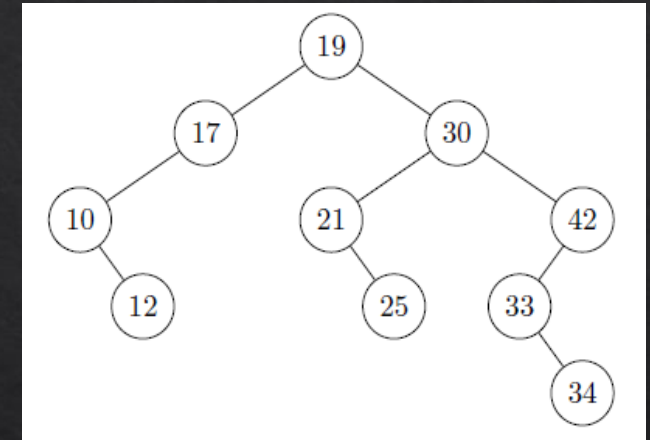
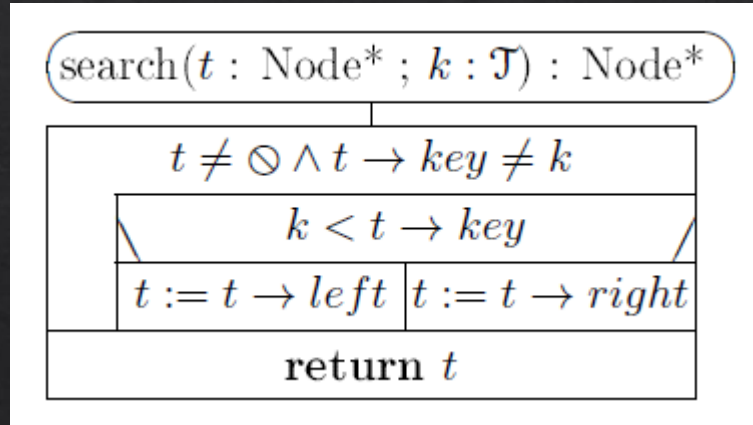
$t = \ominus$			
	$k = t \rightarrow \text{key}$	$k < t \rightarrow \text{key}$	$k > t \rightarrow \text{key}$
return $\ominus$	return t	return $\text{search}(t \rightarrow \text{left}, k)$	return $\text{search}(t \rightarrow \text{right}, k)$



Search

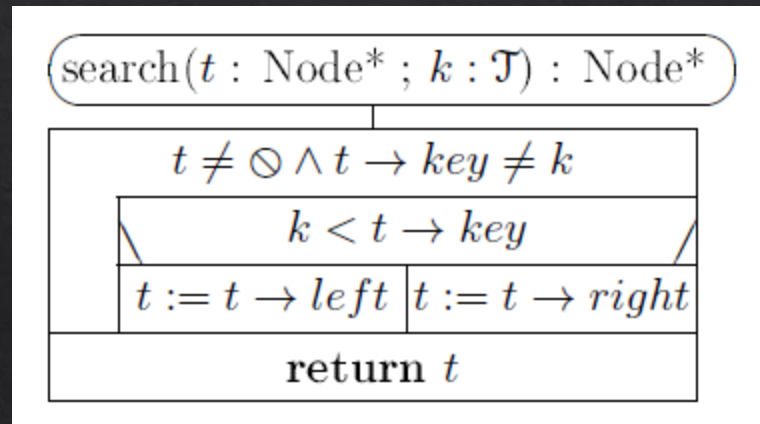
searching for a node with key value k

◇ Iterative algorithm:

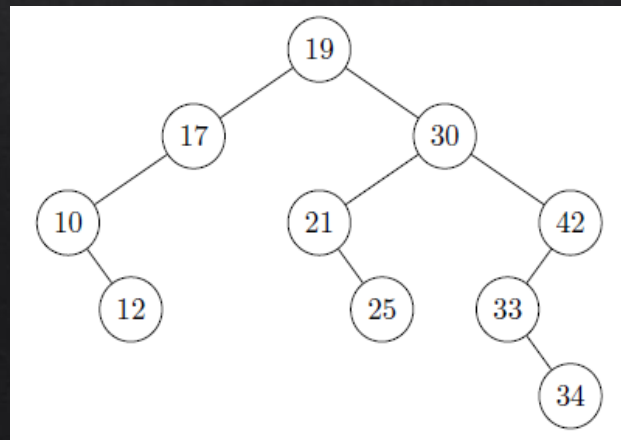
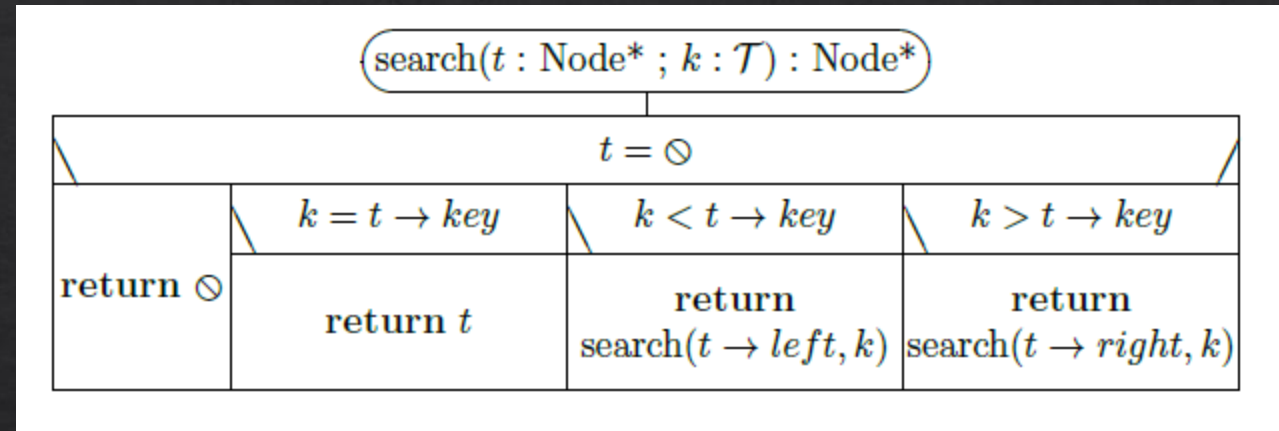


Search

◇ Iterative algorithm:



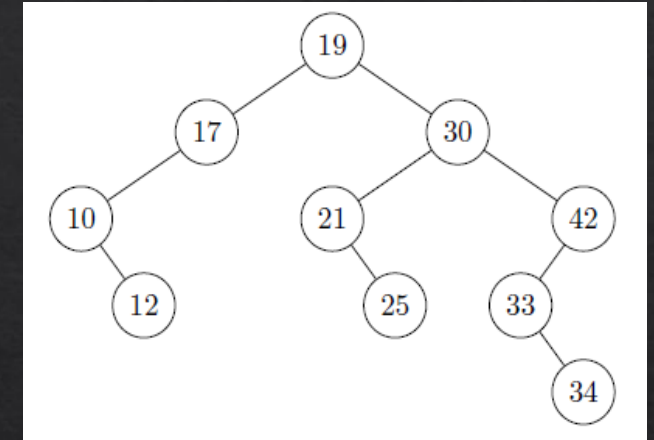
◇ Recursive algorithm:



Insertion

inserting an element with key value k

- First we have to find where k should go.
- If it is there $\rightarrow$ do nothing.
- If not, then create a new node and add.



- Recursive algorithm:

insert($\&t : \text{Node}^* ; k : \mathcal{T}$)			
$t = \emptyset$			
$t :=$	$k < t \rightarrow \text{key}$	$k > t \rightarrow \text{key}$	$k = t \rightarrow \text{key}$
new Node(k)	insert($t \rightarrow \text{left}, k$)	insert($t \rightarrow \text{right}, k$)	SKIP

Insertion

inserting an element with key value k

When we find where it should go, then $p = \text{NIL}$, so we need a pe pointer.

◇ Iterative algorithm:

$\text{insert}(\&t : \text{Node}^* ; k : \mathcal{T})$

$t = \text{NIL}$

$t := \text{new Node}(k)$

$p := t$

$p \neq \text{NIL}$

$pe := p$

$k = p \rightarrow \text{key}$

$k < p \rightarrow \text{key}$

$k > p \rightarrow \text{key}$

return

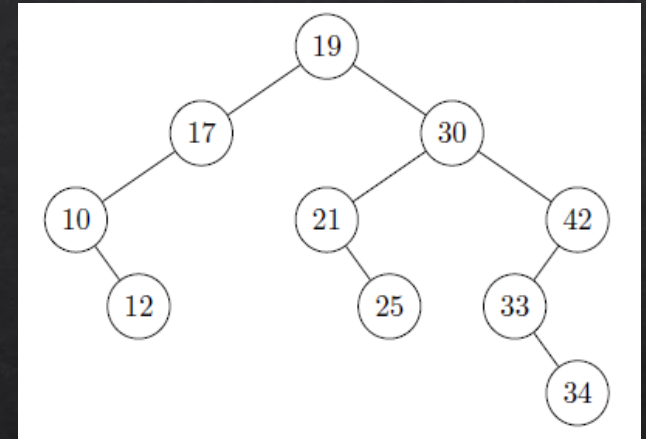
$p := p \rightarrow \text{left}$

$p := p \rightarrow \text{right}$

$k < pe \rightarrow \text{key}$

$pe \rightarrow \text{left} := \text{new Node}(k)$

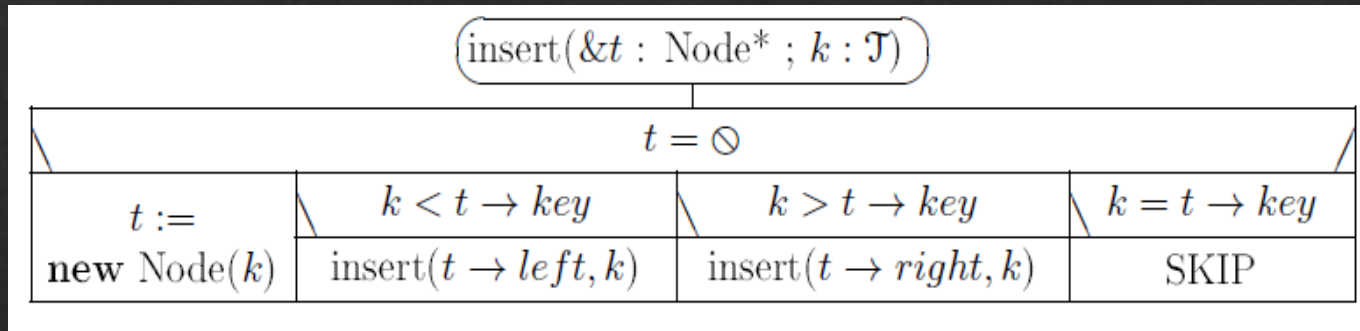
$pe \rightarrow \text{right} := \text{new Node}(k)$



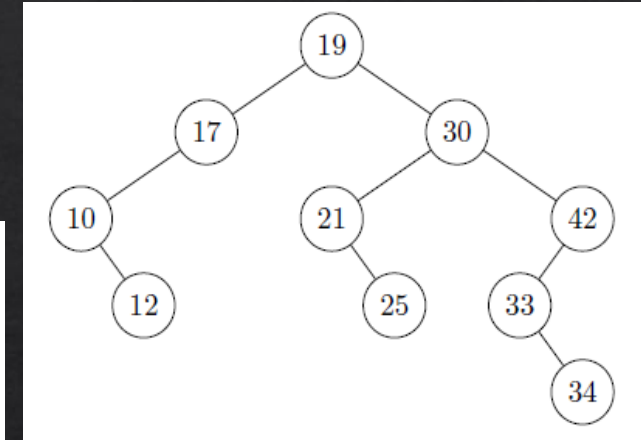
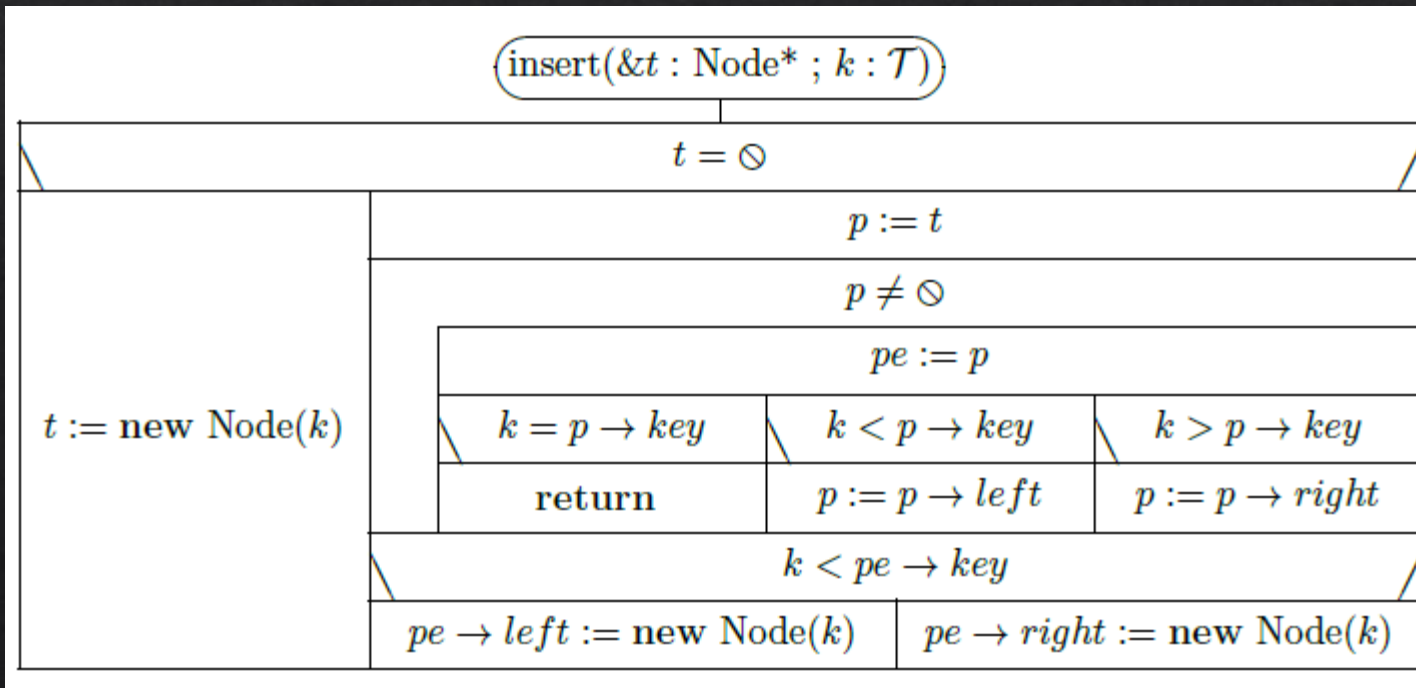
Insertion

inserting an element with key value k

◇ Recursive:



◇ Iterative:

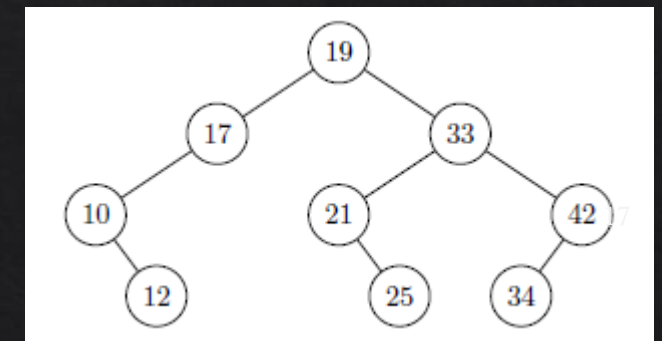
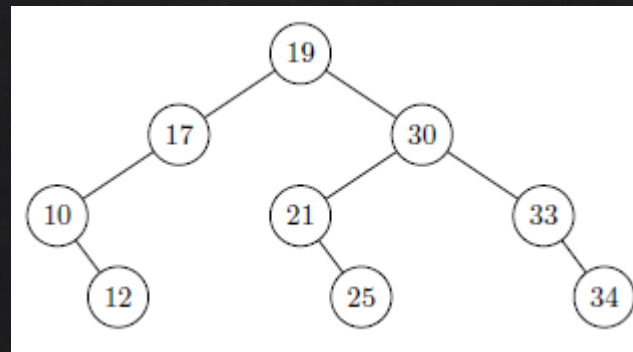
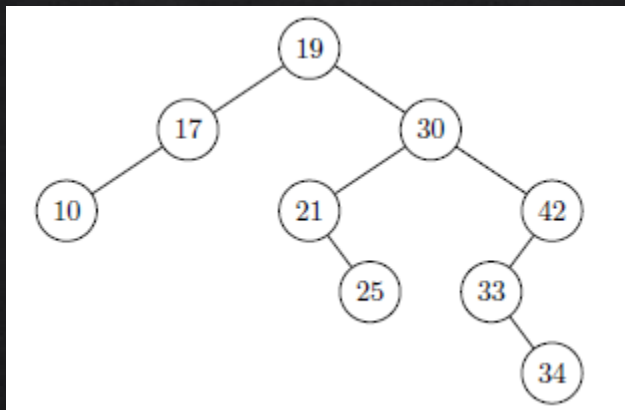
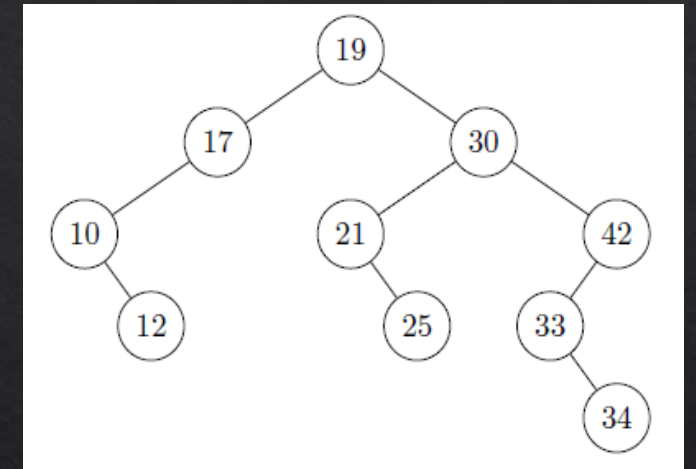


Deletion

deleting a node with key value k

Cases:

- ♦ the node is a leaf: simply remove (example: 12)
- ♦ the node has one child: replace it with child, the subtree comes too (example: 42)
- ♦ the node has two children: we replace it with the node in its **right** subtree that has the **minimum** key value (example: 30 $\rightarrow$ replace it with 33)

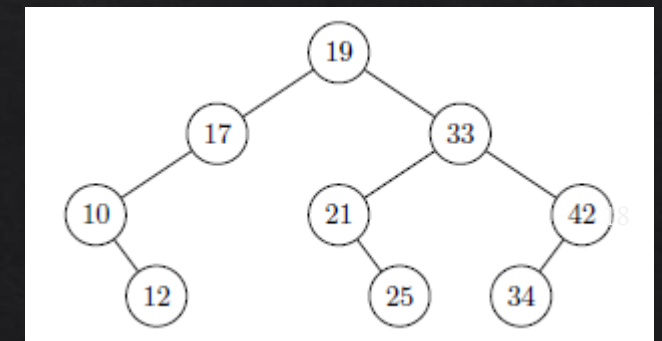
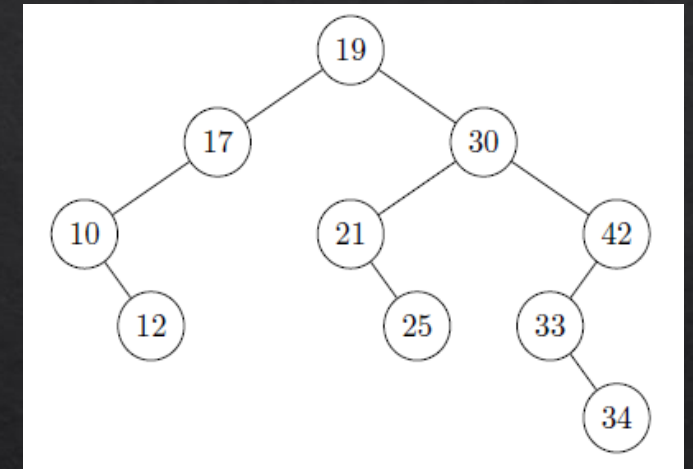


Deletion

deleting a node with key value k

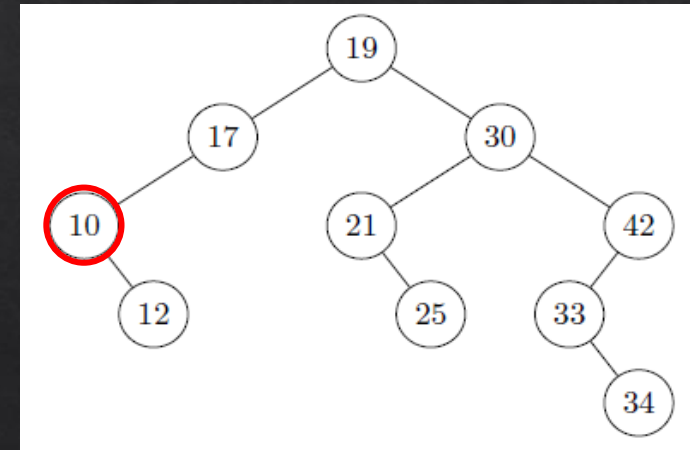
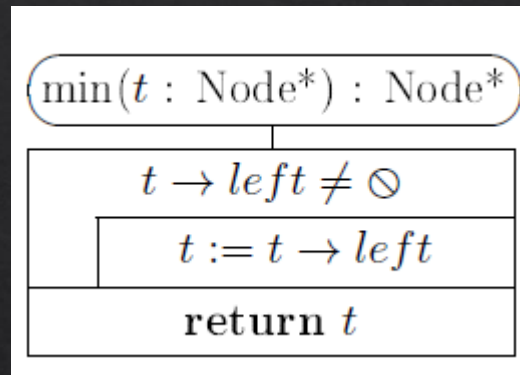
Cases:

- ◆ the node is a leaf: simply remove (example: 12)
- ◆ the node has one child: replace it with child, the subtree comes too (example: 42)
- ◆ the node has two children: we replace it with the node in its **right** subtree that has the **minimum** key value (example: 30)
 - ◆ Where do we find the node with minimum key value?
 - ◆ go as far „left” as you can



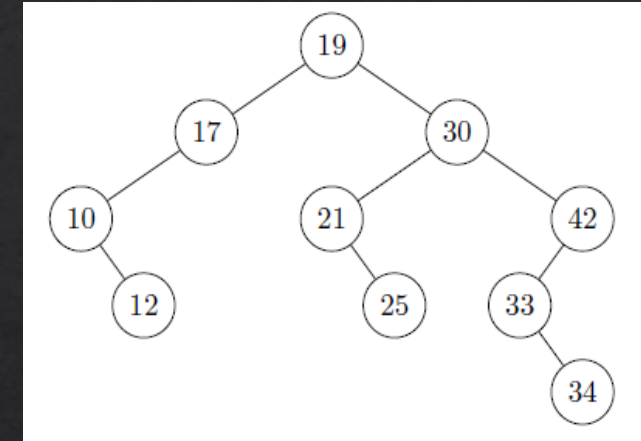
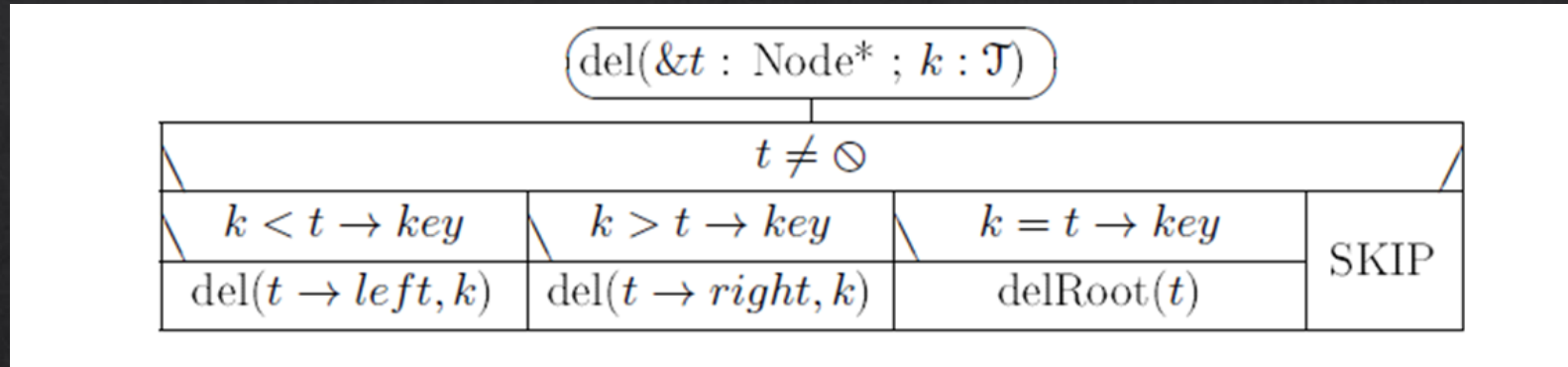
Finding node with minimal key value

- go as far left, as possible



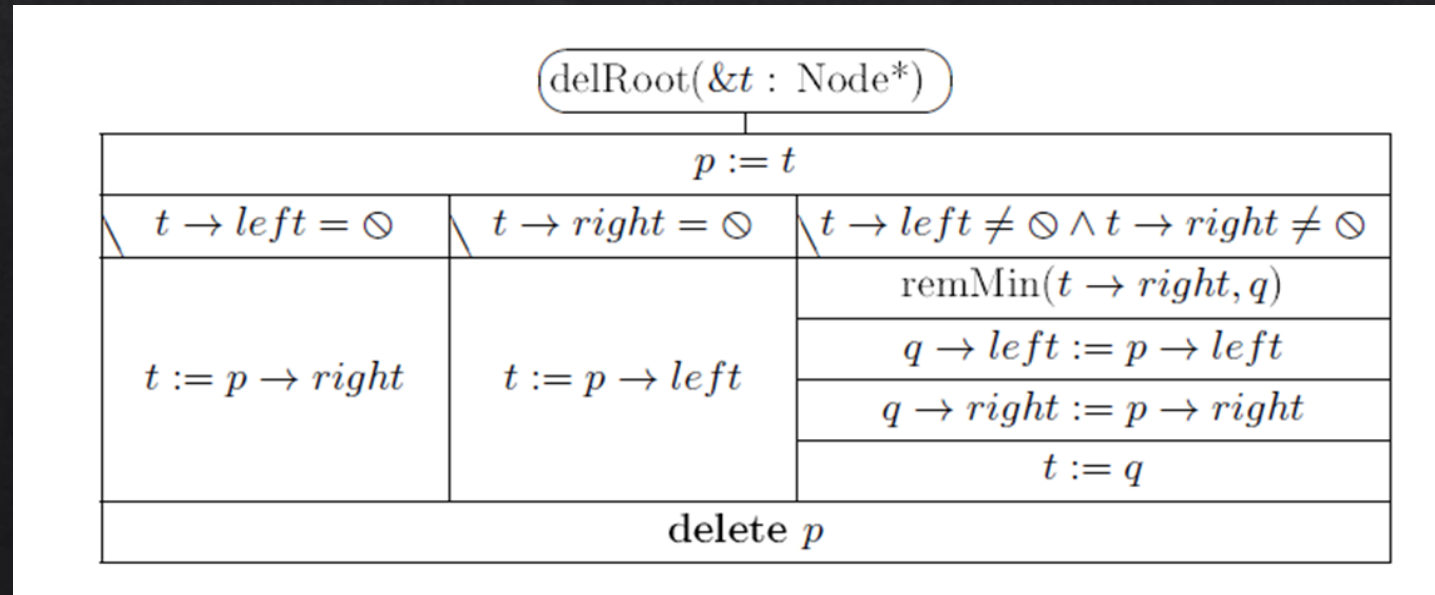
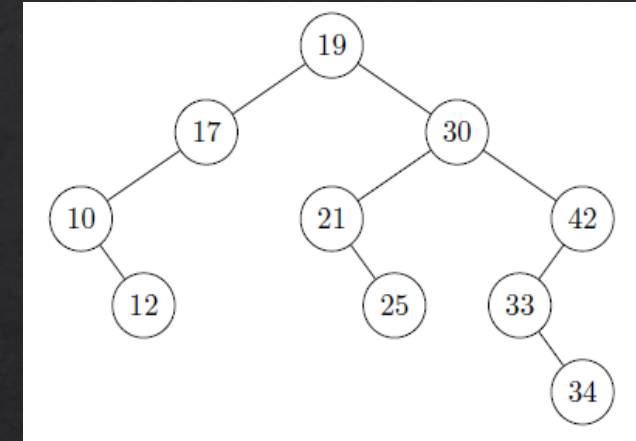
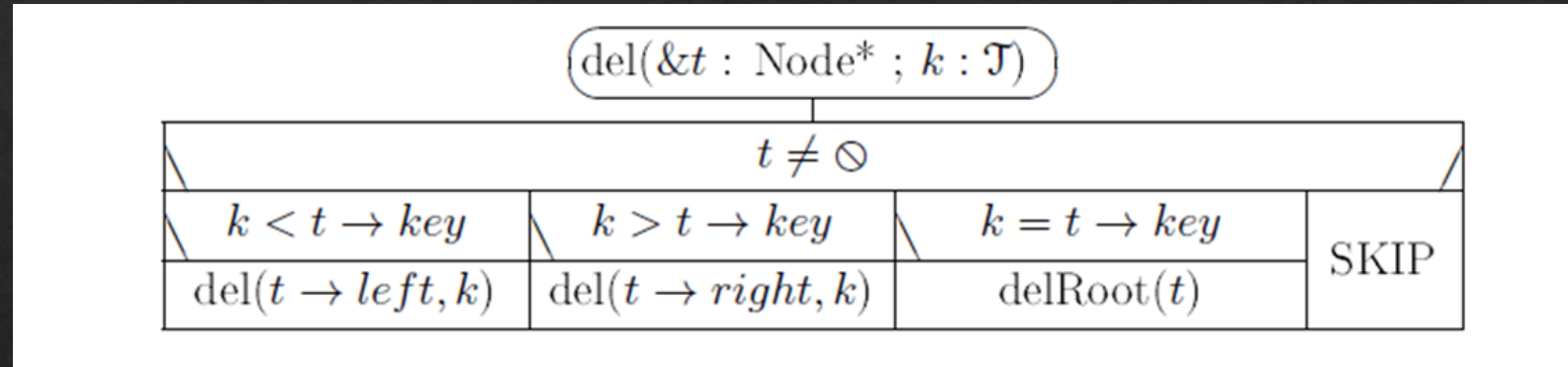
Deletion

First we have to find the node to be deleted, then we have to delete the root from a subtree.



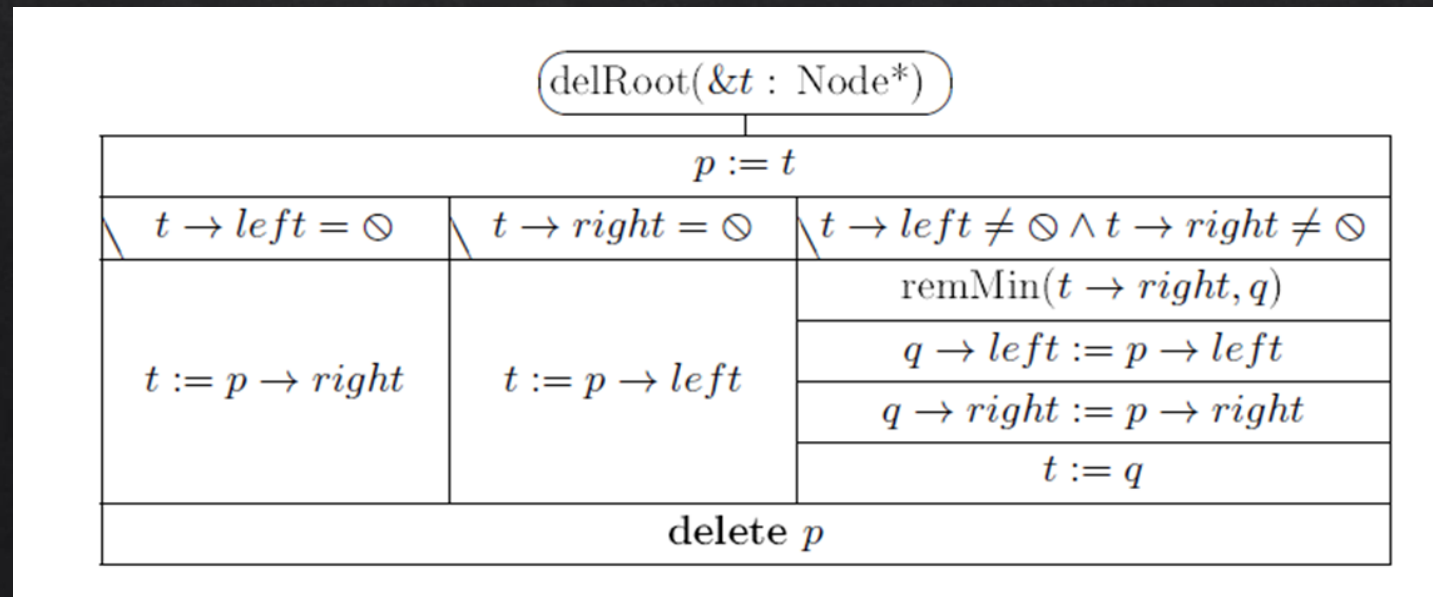
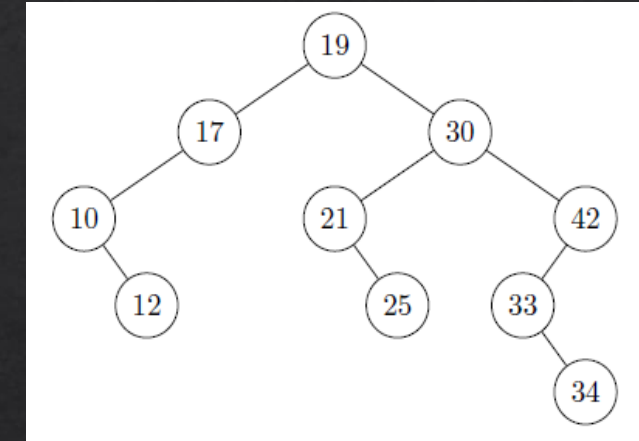
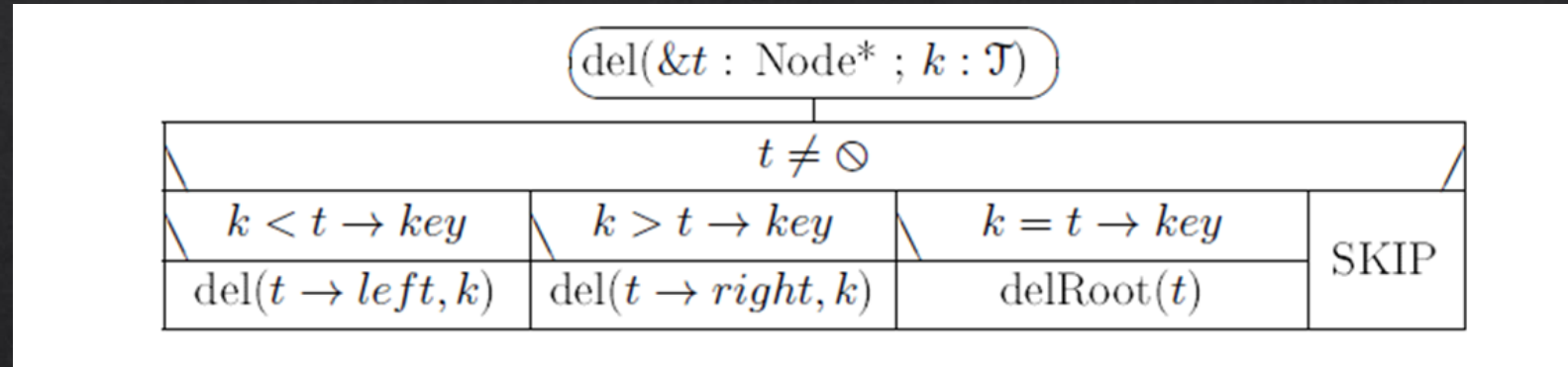
Deletion

First we have to find the node to be deleted, then we have to delete the root from a subtree.



Deletion

First we have to find the node to be deleted, then we have to delete the root from a subtree.



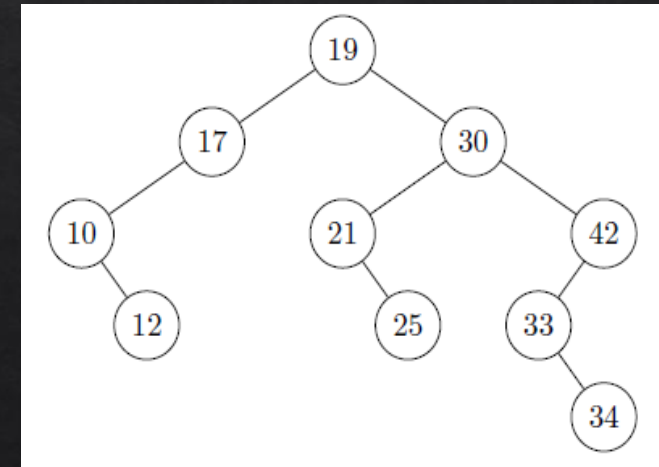
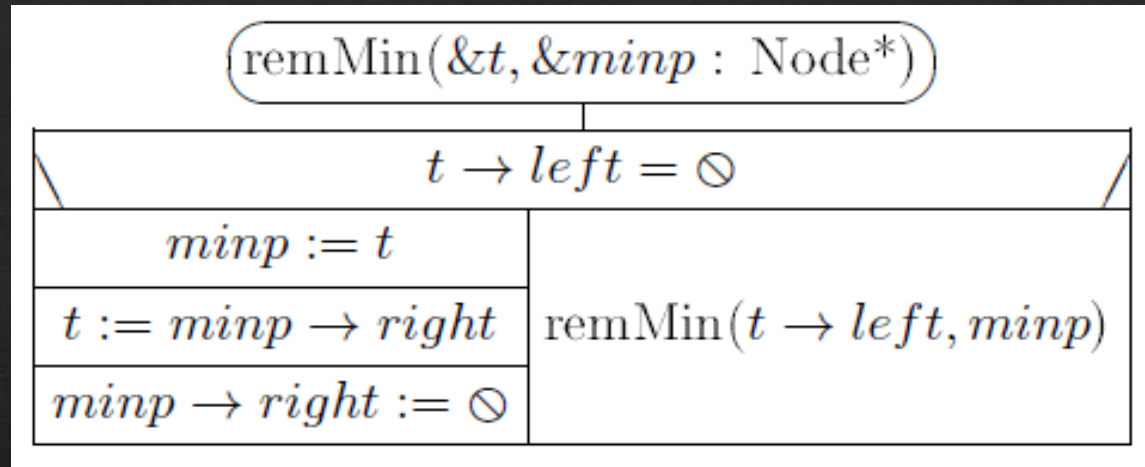
Cases:

- no left child
- no right child
- has 2 children

remMin(t, q): removes the node with minimal key value; q will be a pointer pointing at this node

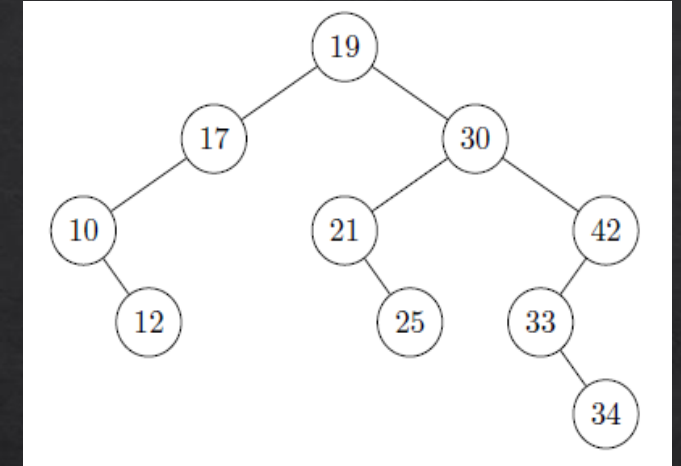
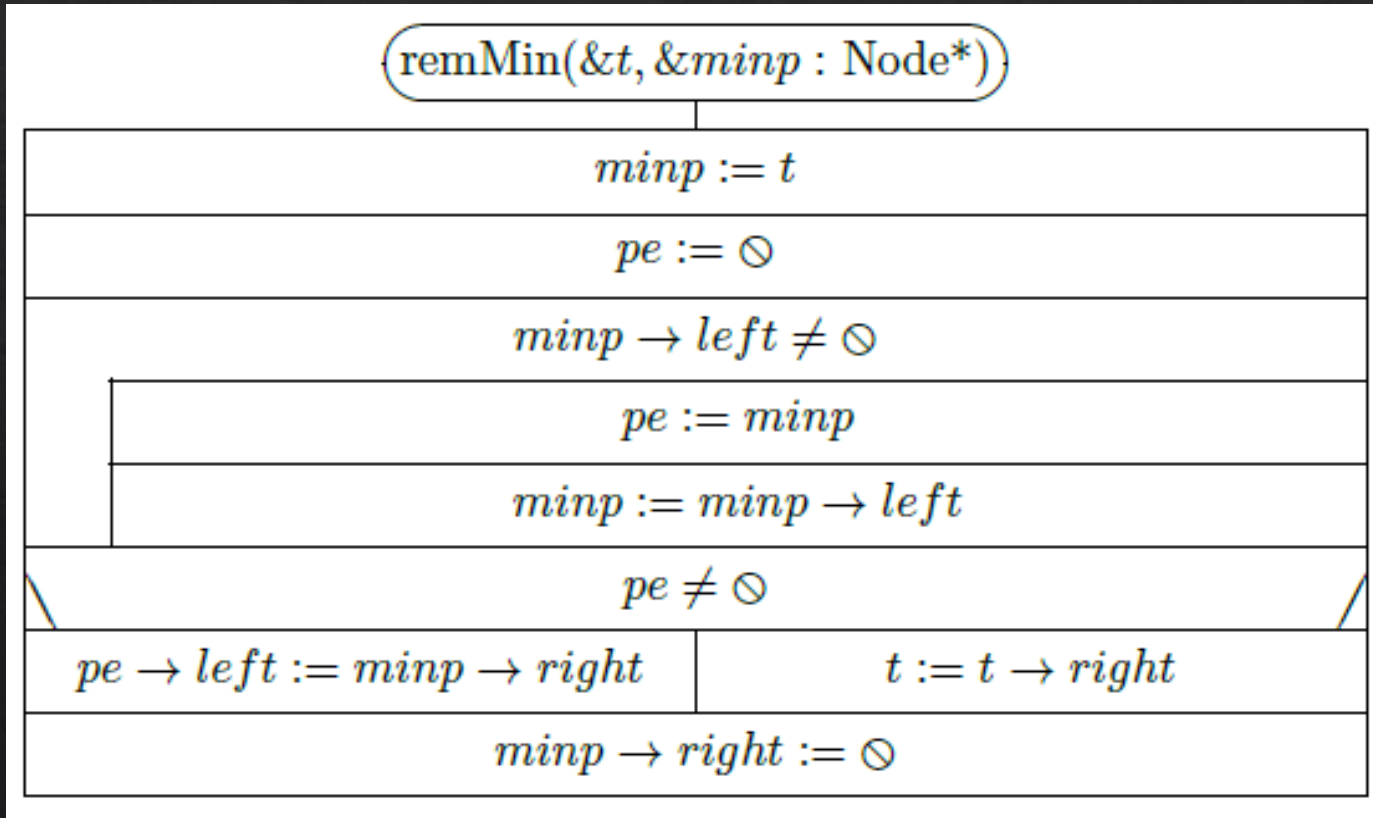
remMin: removing node with minimal key value

◇ Recursive algorithm:



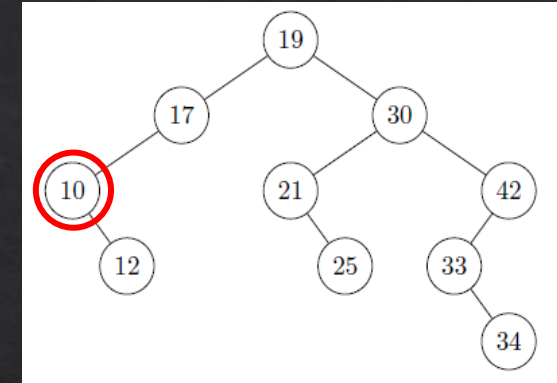
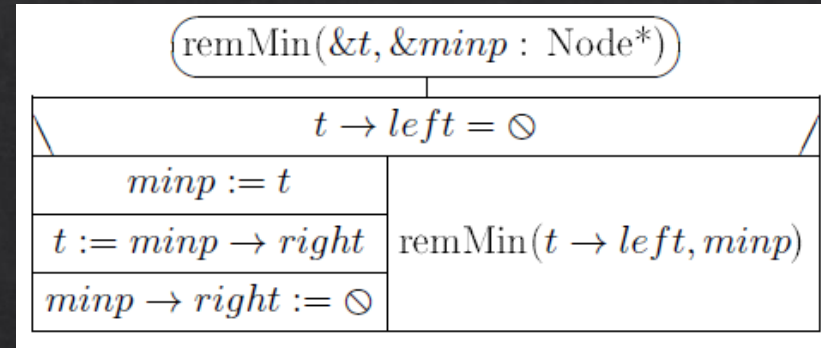
remMin: removing node with minimal key value

◇ Iterative algorithm:

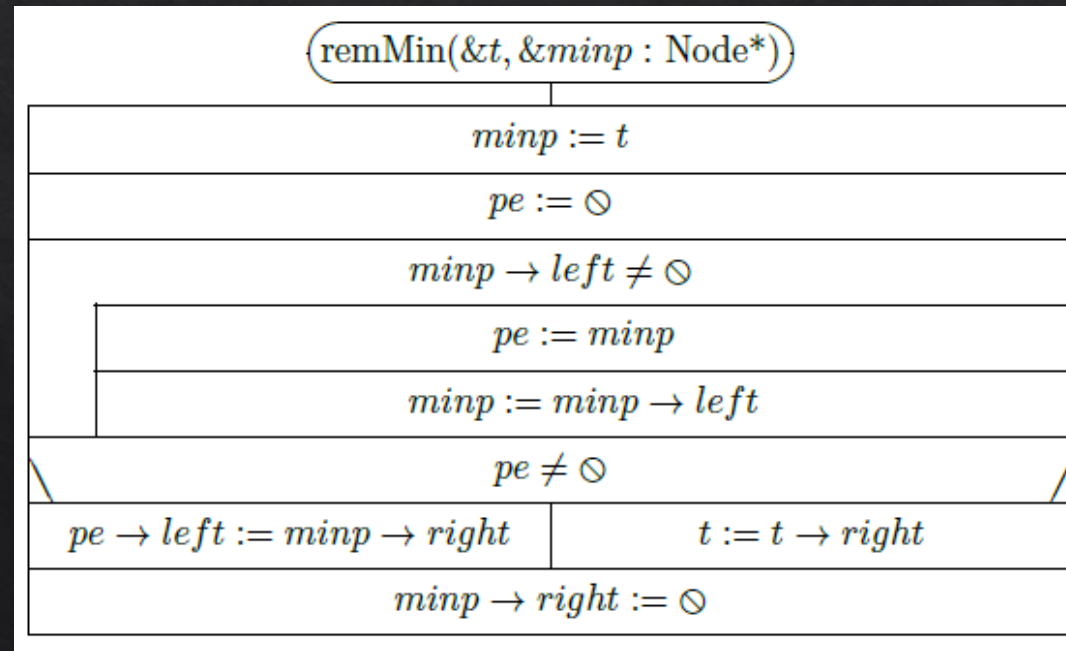


remMin

◇ Recursive:

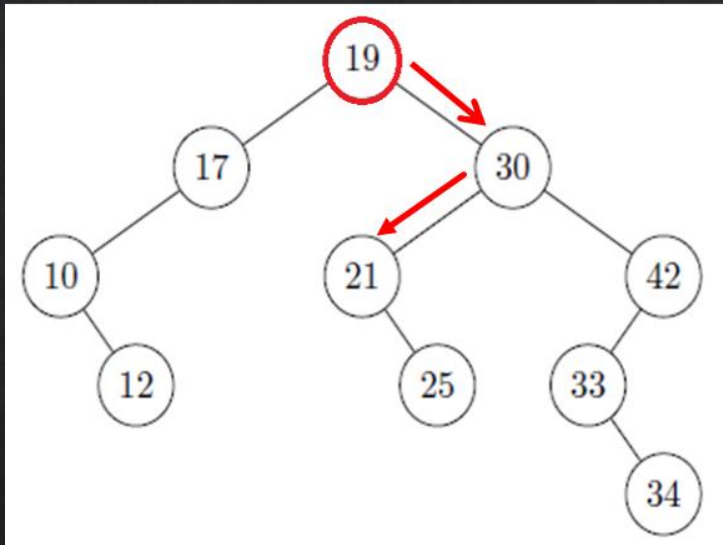


◇ Iterative:

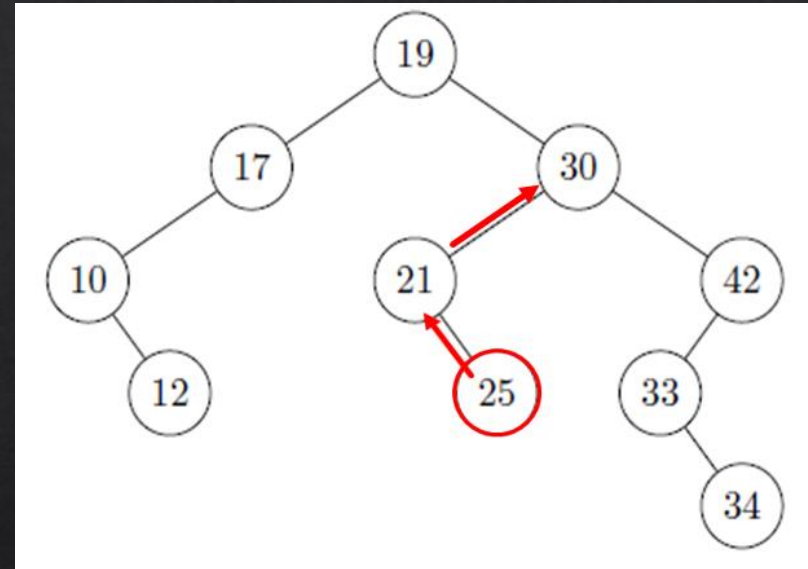


Where is the „successor” of a node in a BST?

- ◆ Given an element, we are looking for the successor, i. e. the next element given by the inorder traversal.
- ◆ For this, we need the parent node as well!

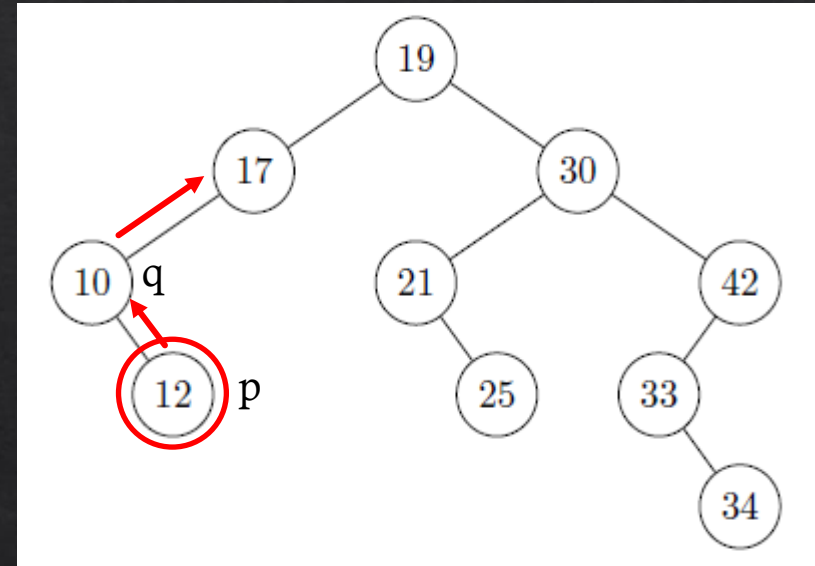
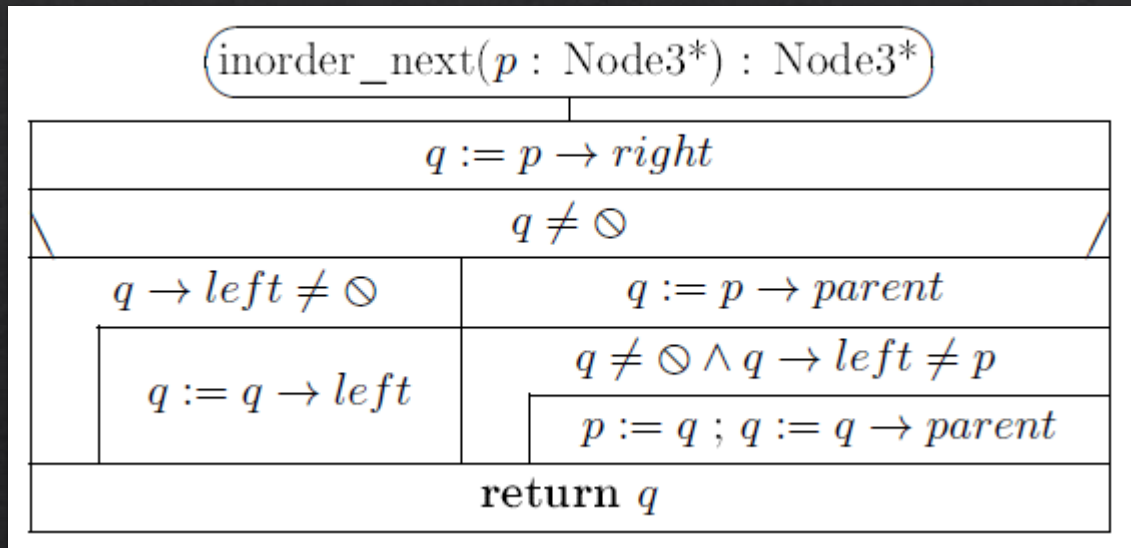


If the given node has a right subtree, then it is the minimum of the right subtree



If the given node doesn't have a right subtree, then we have to look among the ancestors: it will be the first ancestor 'x' for which we are in the left subtree of 'x'.

Where is the „successor” of a node in a BST? for binary trees of Node3 type



You can use this to check if a binary tree is a binary search tree or not.

Assignments

- Design an algorithm which can verify if a given tree is a binary tree or not. (You can use Node3 types, so `t: Node3*` and you can use algorithms from practice.)

Assignment for general binary trees:

- Design an algorithm which uses the level order traversal to count the number of leaves in a binary tree that are on level at least k .